

DOCUMENTATION COMPLETE DU LOGICIEL DE GESTION SCOLAIRE

SOMMAIRE

1. Vue d'ensemble
2. Arborescence complete du projet
3. Architecture et demarrage
4. Point d'entree : `main.py`
5. Configuration : `config.py`
6. Couche donnees : `database/db.py`
7. Depots : `models/repositories.py`
8. Services : `services/`
9. Couche presentation : le dossier `views/`
10. Fichiers `.ui`
11. Base de donnees SQLite : schema complet
12. Roles et permissions
13. Construction des executables
14. Guide d'utilisation par role
15. Depannage et questions frequentes
16. Historique des versions

DOCUMENTATION COMPLETE DU LOGICIEL DE GESTION SCOLAIRE

Version documentee : 1.2.0 — Python 3 + PyQt5 + SQLite

Contexte : Republique du Congo (Brazzaville) — Devise : FCFA
— Indicatif : +242

Ce document decrit chaque fichier du projet, son role, et comment tout s'emboite pour former l'application. Les numeros de lignes correspondent au code source actuel (branche `exe`).

SOMMAIRE

1. Vue d'ensemble
2. Arborescence complete du projet
3. Architecture et demarrage

4. Point d'entree : main.py
 5. Configuration : config.py
 6. Couche donnees : database/db.py
 7. Depots : models/repositories.py
 8. Services : services/
 9. Couche presentation : le dossier views/
 10. Fichiers .ui
 11. Base de donnees SQLite : schema complet
 12. Roles et permissions
 13. Construction des executables
 14. Guide d'utilisation par role
 15. Depannage et questions frequentes
 16. Historique des versions
-

1. Vue d'ensemble

Le logiciel **Gestion Scolaire** est une application de bureau autonome pour un etablissement scolaire congolais. Elle couvre :

- la **gestion des eleves** (inscription, reinscription, matricule automatique, dossier complet) ;
- la **gestion des classes** (capacite, salle, titulaire) ;
- la **saisie des notes** et la **generation de bulletins** ;
- l'**emploi du temps** (planning hebdomadaire) ;
- la **caisse** (recettes / depenses, solde, export CSV) ;
- la **gestion des comptes utilisateurs** (3 roles) ;
- la **gestion du personnel** (RH, salaires, paie) ;
- la **generation de documents** (bulletins, recus, certificats, paie, rapports RH, emploi du temps) au format **HTML** ouverts dans le navigateur ;
- des **dashboards** par role avec graphiques (barres, camemberts) dessines sans aucune dependance externe grace a `QPainter` .

Caracteristiques techniques

Element	Valeur
Langage	Python 3
Interface graphique	PyQt5 (5.15.11)
Base de donnees	SQLite (fichier <code>ecole.db</code>)

Element	Valeur
Backend optionnel	API REST FastAPI (<code>http://127.0.0.1:8000</code>) pour enrichir les indicateurs (voir section 8.2)
Mode hors ligne	L'app fonctionne complètement sans l'API : bascule sur la base locale
Empaquetage	PyInstaller one-file sur Linux et Windows
Dependances	<code>PyQt5==5.15.11</code> et <code>requests==2.32.3</code> (voir <code>requirements.txt</code>)

Où sont stockées les données ?

- En développement : dossier `data/` à la racine du projet.
- En exécutable : dossier `data/` **à côté de l'exécutable** (voir `config.data_dir()`).

2. Arborescence complète du projet

```

Logiciel-de-gestion-scolaire/
|
├─ main.py                # Point d'entree : HighDPI + demarrage (66 lignes)
├─ config.py              # Constantes + feuille de style (118 lignes)
├─ requirements.txt       # PyQt5==5.15.11 + requests==2.32.3
├─ .gitignore             # venv/, .venv/, __pycache__/, *.db, build/, dist/,
dist_win/, exe-win/, data/, GestionScolaire.spec
|
├─ build_win.spec          # Spec PyInstaller WINDOWS (one-file + manifeste DPI)
├─ win_dpi_manifest.xml    # Manifeste <dpiAware>true</dpiAware> pour Windows
|
├─ .github/workflows/
|   └─ build_windows.yml    # Workflow GitHub Actions : build de l'exe Windows
|
├─ database/
|   ├── __init__.py        # Importe l'instance partagée de la base
|   └─ db.py               # Connexion SQLite, schema, données initiales (234 lignes)
|
├─ models/
|   ├── __init__.py        # Importe l'instance partagée des dépôts
|   └─ repositories.py     # Toutes les requêtes métier (300 lignes)
|
├─ services/
|   ├── __init__.py        # Vide
|   ├── api.py             # Client de l'API FastAPI + repli local (386 lignes)
|   ├── auth.py            # Connexion, mots de passe, permissions (69 lignes)
|   └─ reports.py          # Documents HTML (bulletins, certificats, paie) (193
lignes)
|
├─ views/
|   ├── __init__.py        # Vide
|   ├── loader.py          # Chargement des .ui + responsivité (26 lignes)
|   ├── login_view.py      # Fenêtre de connexion (125 lignes)
|   ├── main_view.py       # Fenêtre principale, sidebar, statut API (191 lignes)
|   └─ pages.py            # Toutes les pages et dialogues métier (1455 lignes)

```

```

|   ├── widgets.py           # Graphiques QPainter + formats monetaires (176 lignes)
|   |
|   └── ui_files/            # Interfaces .ui (Qt Designer)
|       ├── main.ui          # Fenetre principale (sidebar + QStackedWidget)
|       ├── dashboards/      # dashboard_admin.ui, dashboard_directeur.ui,
|       |                     # dashboard_gestionnaire.ui
|       ├── eleves/          # eleves.ui (liste) + inscription.ui (dossier)
|       ├── classes/         # classes.ui + classe_dialog.ui
|       ├── notes/           # notes.ui (saisie + bulletins)
|       ├── planning/        # planning.ui (emploi du temps)
|       ├── caisse/          # caisse.ui (recettes / depenses)
|       ├── comptes/         # comptes.ui + compte_dialog.ui
|       └── parametres/      # parametres.ui (en-tete des documents)
|
|   ├── data/                # CREE A L'EXECUTION (base + documents)
|       ├── ecole.db          # La base SQLite
|       └── documents/        # HTML generes (bulletins, recus, etc.)
|
|   ├── dist/GestionScolaire  # Executable Linux (one-file)
|   ├── dist_win/             # Executable Windows (one-file, via GitHub Actions)
|   ├── build/                 # Travail intermediaire PyInstaller (nettoyable)
|   └── .venv/                 # Environnement virtuel de developpement

```

3. Architecture et demarrage

L'application suit une architecture en **3 couches**, sans framework :

```

+-----+
| COUCHE PRESENTATION  (views/) |
| login_view.py -> main_view.py -> pages.py -> widgets.py |
| fichiers .ui (views/ui_files) |
+-----+
| importe / appelle
+-----v-----+
| COUCHE SERVICES  (services/) |
| auth.py (connexion, permissions) |
| api.py (client API REST + repli local) |
| reports.py (documents HTML) |
+-----+
| utilise
+-----v-----+
| COUCHE DONNEES  (models/ + database/) |
| models/repositories.py (requetes metier) |
| database/db.py (connexion, schema, seed) |
| SQLite : data/ecole.db |
+-----+

```

Demarrage pas a pas

1. `python main.py` est execute.

2. `main()` (`main.py:44`) :

- configure le **HighDPI** (`_setup_high_dpi`, `main.py:16`) ;
- choisit une police vectorielle (`_pick_base_font`, `main.py:24`) ;
- installe `_excepthook` (`main.py:35`) : boîte de dialogue au lieu d'un crash silencieux en executable ;
- cree l'application Qt et applique `APP_STYLESHEET` (`config.py:29`) ;
- appelle `db.init_db()` -> creation des tables + donnees initiales ;
- ouvre la **fenetre de connexion** `LoginDialog` ;
- si la connexion reussit -> ouvre `MainWindow` en plein ecran ;
- sinon -> quitte.

Navigation dans la fenetre principale

- `MainWindow` charge `main.ui` : une **sidebar** de boutons et un `QStackedWidget` pour le contenu.
- `_wire_nav()` (`main_view.py:128`) affiche uniquement les boutons autorises pour le role connecte.
- `_build_pages()` (`main_view.py:138`) construit chaque page autorisee dans un `QWidget` entoure d'une `QScrollArea`, et l'ajoute au `QStackedWidget`. En cas d'erreur, un libelle rouge s'affiche au lieu de faire planter l'app.
- `navigate(page)` (`main_view.py:173`) bascule la page, appelle son `refresh()` et coche le bouton actif.

Rafraichissement

Chaque page expose `page.refresh = refresh`. A chaque navigation, `MainWindow.navigate` appelle `refresh()` pour recharger les donnees depuis la base. Les pages restent en memoire et se rafraichissent a la demande.

4. Point d'entree : `main.py`

`main.py` (66 lignes)

Ligne	Element	Role
16-22	<code>_setup_high_dpi()</code>	Rendu net sur les ecrans Windows 100/125/150/200 % (variables d'environnement Qt posees avant <code>QApplication</code>).

Ligne	Element	Role
24-33	<code>_pick_base_font()</code>	Police vectorielle presente sur le systeme (<code>APP_FONT_FAMILY</code> , puis fallbacks), sinon police generique.
35-42	<code>_excepthook(...)</code>	En executable, une exception non geree affiche une boite « Erreur inattendue » avec le detail.
44-63	<code>main()</code>	Orchestration complete du demarrage (voir section 3).
65-66	garde	<code>if __name__ == "__main__": main()</code>

5. Configuration : `config.py`

`config.py` (118 lignes)

Module importe partout : constantes, chemins, feuille de style.

Constantes principales

Constante	Valeur	Ligne	Role
<code>APP_NAME</code>	"Gestion Scolaire"	6	Titre de l'application
<code>APP_VERSION</code>	"1.2.0"	7	Version affichee sur l'ecran de connexion
<code>APP_FONT_FAMILY</code> / <code>APP_FONT_FALLBACK</code> / <code>APP_FONT_SIZE</code>	"Segoe UI" / tuples / 10	14-16	Police et taille du texte
<code>API_BASE_URL</code>	"http://127.0.0.1:8000"	19	Adresse du backend FastAPI (optionnel)
<code>API_TIMEOUT</code>	3.0	20	Delai d'attente avant bascule sur la base locale
<code>PAYS_DEFAULT</code> / <code>VILLE_DEFAULT</code>	"Republique du Congo" / "Brazzaville"	23-24	Valeurs par default des documents
<code>INDICATIF_TEL</code> / <code>DEVISE</code>	" +242 " / "FCFA"	25-26	Contexte congolais
<code>APP_STYLESHEET</code>	CSS Qt	29-46	Feuille de style globale de l'app
<code>ROLES</code>	("admin", "directeur", "gestionnaire")	49	Roles possibles
<code>ROLE_LABELS</code>	{"admin": "Administrateur", ...}	51	Libelles affiches
<code>PERIODES</code>	("1er Trimestre", ...)	58	Periodes de notes
<code>JOURS</code>	("Lundi", ..., "Samedi")	60	Jours du planning
<code>CRENEAUX</code>	8 creneaux	62	Horaires des cours
<code>DEFAULT_ACCOUNTS</code>	3 comptes	94	Comptes crees au premier lancement (voir section 11)
<code>DEFAULT_MATIERES</code>	8 matieres	107	

Constante	Valeur	Ligne	Role
			Francais, Maths, Anglais, HG, SVT, PC, Educ. civique, Informatique

Fonctions de chemins

Fonction	Ligne	Role
<code>resource_path(relative)</code>	74	Chemin vers les donnees empaquetees dans l'executable (<code>_MEIPASS</code>), sinon racine du projet.
<code>data_dir()</code>	80	Dossier ecrivable ou vit la base : a cote de l'executable, sinon racine du projet. Cree <code>data/</code> si besoin.

Constantes de chemins

Constante	Ligne
<code>UI_DIR</code> (dossier des <code>.ui</code>)	89
<code>DB_PATH</code> (fichier <code>data/ecole.db</code>)	90
<code>DOCS_DIR</code> (dossier <code>data/documents/</code>)	91

En fin de module, `os.makedirs(data_dir(), exist_ok=True)` garantit que le dossier `data/` existe des le chargement de `config`.

6. Couche donnees : `database/db.py`

`database/db.py` (234 lignes)

Fonctions et schema

Element	Ligne	Role
<code>SCHEMA</code>	7	Script SQL de creation des tables (voir section 11).
<code>hash_password(password)</code>	121	Retourne le hash <code>sha256</code> hexadecimal. Les mots de passe ne sont jamais stockes en clair.

Classe `Database` (singleton)

Une seule instance partagee par toute l'app.

Methode	Ligne	Role
<code>__new__</code>	129	Retourne l'unique instance
<code>__init__</code>	134	Prepare le dossier des documents, pose <code>_initialized = False</code>
<code>connect()</code>	139	Ouvre une connexion SQLite (<code>row_factory = Row</code> -> dictionnaires), active <code>PRAGMA foreign_keys = ON</code> . Une connexion par operation.

Methode	Ligne	Role
<code>init_db()</code>	147	Execute <code>SCHEMA</code> puis <code>_seed</code> , une seule fois.
<code>_seed(conn)</code>	160	Comptes par default (si aucun utilisateur), matieres, parametres, 3 membres de personnel de base.
<code>query(sql, params)</code>	204	SELECT -> liste de dictionnaires.
<code>query_one(sql, params)</code>	213	Premier enregistrement ou <code>None</code> .
<code>execute(sql, params)</code>	218	INSERT/UPDATE/DELETE, commit, retourne <code>lastrowid</code> .
<code>executemany(sql, seq)</code>	228	Execution en lot, commit.

Le `_seed` cree uniquement les donnees de base (comptes, matieres, parametres, personnel). Les eleves, transactions et notes sont saisis par l'utilisateur dans l'application.

7. Depots : `models/repositories.py`

`models/repositories.py` (300 lignes)

La classe `Repositories` regroupe **toutes les requetes metier**. Elle est importee partout sous l'alias `repos` (`from models import repos`).

Helper

Fonction	Ligne	Role
<code>_gen_reference(prefix)</code>	9	Reference unique de transaction <code>REC-XXXXXXX</code> / <code>DEP-XXXXXXX</code> .

Methodes par domaine

Classes & matieres

Methode	Ligne	Role
<code>classes()</code>	15	Toutes les classes + effectif reel (COUNT sur <code>eleves</code>), trieés par nom.
<code>classe_by_id(id)</code>	20	Une classe.
<code>add_classe(...)</code>	23	Insere une classe.
<code>update_classe(...)</code>	28	Modifie une classe.
<code>delete_classe(id)</code>	33	Supprime d'abord ce qui depend de la classe (eleves, planning), puis la classe.
<code>matieres()</code>	39	Toutes les matieres trieés.

Methode	Ligne	Role
<code>add_matiere(nom)</code>	42	Ajoute une matiere (IGNORE si doublon).

Eleves

Methode	Ligne	Role
<code>eleves(classe_id, statut, recherche)</code>	45	Liste filteree (classe, statut, texte) avec <code>classe_nom</code> via LEFT JOIN.
<code>eleve_by_id(id)</code>	63	Un eleve.
<code>eleve_by_matricule(m)</code>	66	Un eleve par matricule (reinscription).
<code>add_eleve(data)</code>	69	Insere un eleve ; genere le matricule si absent.
<code>update_eleve(id, data)</code>	87	Met a jour le dossier.
<code>delete_eleve(id)</code>	98	Supprime notes puis eleve.
<code>next_matricule()</code>	102	Prochain matricule <code>ELEV{annee}{NNNN}</code> a partir du dernier existant.

Notes

Methode	Ligne	Role
<code>notes_for(classe_id, matiere_id, periode)</code>	118	Notes d'une classe x matiere x periode (avec matricule et nom).
<code>save_note(...)</code>	127	Insere ou met a jour (UPSERT) sur <code>(eleve_id, matiere_id, periode)</code> .

Caisse

Methode	Ligne	Role
<code>transactions(type, recherche, dates)</code>	137	Liste filteree, tri date DESC.
<code>add_transaction(...)</code>	158	Insere avec reference auto <code>REC-... / DEP-...</code>
<code>delete_transaction(id)</code>	167	Supprime.
<code>caisse_totals()</code>	170	Somme des entrees, des sorties, solde.
<code>export_transactions_csv(path)</code>	180	CSV separe par <code>;</code> avec BOM (<code>utf-8-sig</code>) pour Excel.

Comptes utilisateurs

Methode	Ligne	Role
<code>utilisateurs(role, recherche)</code>	189	Liste (sans le hash de mot de passe).
<code>add_compte(...)</code>	203	Cree un compte ; username derive de l'email.
<code>update_compte(...)</code>	210	Modifie identite/email/telephone/role/actif.
<code>toggle_compte(id, actif)</code>	216	Active/desactive.
<code>reset_password(id, hash)</code>	219	Remplace le mot de passe.
<code>delete_compte(id)</code>	222	Supprime connexions puis compte.

Planning

Methode	Ligne	Role
<code>planning_for(classe_id)</code>	226	Dictionnaire {(jour, creneau): entree}.
<code>save_planning(classe_id, entries)</code>	232	Remplace tout le planning d'une classe.

Personnel

Methode	Ligne	Role
<code>personnel(recherche)</code>	238	Liste du personnel.
<code>add_personnel(...)</code>	247	Ajoute.
<code>update_personnel(...)</code>	253	Modifie.
<code>delete_personnel(id)</code>	259	Supprime.
<code>masse_salariale()</code>	262	Somme des salaires.
<code>enseignants()</code>	266	Personnel dont la fonction contient « Enseignant ».

Parametres

Methode	Ligne	Role
<code>parametres()</code>	269	Dictionnaire cle -> valeur.
<code>set_parametre(cle, valeur)</code>	274	Upsert d'un parametre.
<code>delete_parametres()</code>	279	Reinitialise signataire, titre, ville.

Statistiques du dashboard

Methode	Ligne	Role
<code>stats_dashboard()</code>	284	Total eleves, inscriptions du jour, encaissements du jour, dossiers incomplets.

8. Services : services/

8.1 services/auth.py (69 lignes) — connexion et permissions

Classe AuthService

Methode	Ligne	Role
<code>login(username, password)</code>	10	Recherche par username ou email ; verifie activite puis mot de passe (hash) ; met a jour <code>last_login</code> et journalise la connexion. Retourne <code>(user, None)</code> ou <code>(None, message)</code> .
<code>change_password(user_id, old, new)</code>	27	Verifie l'ancien mot de passe puis met a jour.
<code>random_password()</code>	36	Mot de passe temporaire aleatoire (12 caracteres hexa).

Methode	Ligne	Role
derniere_connexions(limit=20)	40	Journal des connexions (nom, role, date).

Classe RoleAuthorizer (permissions par role)

- NAV (ligne 48) : pages autorisees par role :
 - admin : dashboard , comptes ;
 - directeur : dashboard , eleves , classes , notes , planning , caisse , personnel , parametres ;
 - gestionnaire : dashboard , eleves , classes , notes , planning , caisse .
- allowed(page) (60) : la page est-elle autorisee ?
- can_edit(page) (64) : qui peut saisir ? comptes -> seul admin ; sinon -> le gestionnaire oui, les autres non.

8.2 services/api.py (386 lignes) — client de l'API FastAPI

L'app peut afficher les indicateurs depuis un backend REST FastAPI (`http://127.0.0.1:8000`). Si l'API est hors ligne, elle **bascule automatiquement sur la base locale** : l'utilisateur ne voit aucune erreur.

Element	Ligne	Role
ApiError	11	Exception interne (jamais exposee a l'interface).
_request(method, path, **kwargs)	16	Envoie une requete HTTP et normalise (donnees, erreur) . Toutes les erreurs reseau sont interceptees.
_CacheDispo	41	Memorise pendant 5 s si l'API repond (evite de tester a chaque appel).
api_disponible(force=False)	57	L'API est-elle joignable ?
ApiClient	64	Un client par domaine : eleves/classes, paiements, enseignants, notes, presences, annees scolaires. Chaque methode renvoie (donnees, erreur) .
client	386	Instance unique utilisee par les dashboards.

Exemples de methodes : `total_eleves()` , `eleve_recherche(...)` , `total_montant_paiement()` , `paiements()` , `enseignants()` , `ajouter_eleve(...)` .

La barre d'etat de la fenetre principale affiche l'etat de la connexion : « **API : connectee** » ou « **API : hors ligne (mode local)** » (main_view.py:113-118).

8.3 services/reports.py (193 lignes) — documents HTML

Tous les documents sont generes en **HTML** dans `DOCS_DIR` puis **ouverts dans le navigateur** (`QDesktopServices.openUrl`). Pas de dependance PDF.

Element	Ligne	Role
<code>STYLE</code>	9	CSS des documents (tableaux bordes, en-tete vert #047857).
<code>_open_in_browser(path)</code>	19	Ouvre un fichier local dans le navigateur.
<code>_write(title, body, filename)</code>	25	Ecrit le HTML complet et l'ouvre.
<code>_entete_doc()</code>	34	En-tete generique (nom du logiciel, pays/ville, date).
<code>export_eleves_csv(eleves)</code>	45	CSV (; , BOM) dans <code>data/</code> .
<code>bulletins(classe_id, periode)</code>	59	Bulletins d'une classe. Moyenne par matiere : $(d1 + d2 + 2 \times comp) / 4$, moyenne generale ponderee.
<code>recu_paiement(...)</code>	98	Recu de paiement.
<code>certificat_scolarite(eleve, params)</code>	117	Certificat avec le signataire (parametres).
<code>paie()</code>	137	Bulletins de paie du mois + masse salariale.
<code>rapport_rh()</code>	152	Rapport RH mensuel.
<code>planning(classe)</code>	165	Emploi du temps d'une classe.
<code>_appreciation(moyenne)</code>	181	Mention : ≥ 16 Excellent, ≥ 14 Tres bien, ≥ 12 Bien, ≥ 10 Assez bien, ≥ 8 Passable, sinon Insuffisant.

9. Couche presentation : le dossier `views/`

9.1 views/loader.py (26 lignes)

Fonction	Ligne	Role
<code>_make_responsive(widget)</code>	8	Applique <code>QSizePolicy.Expanding</code> uniquement a la page racine (volontaire : l'expansion forcee sur les enfants etire les boutons en blocs geants).
<code>load_ui(relative_path, widget=None)</code>	18	Charge un <code>.ui</code> via <code>PyQt5.uic.loadUi</code> , rend la page responsive.
<code>apply_ui(relative_path, widget)</code>	24	Applique un <code>.ui</code> a un widget existant (MainWindow, dialogues).

9.2 views/main_view.py (191 lignes)

Element	Ligne	Role
<code>NAV_PAGES</code>	9	

Element	Ligne	Role
		Correspondance bouton sidebar -> page (dashboard, eleves, caisse, classes, notes, planning, personnel, parametres, comptes).
DASHBOARD_BUILDERS	21	role -> fabrique de dashboard (admin/directeur/gestionnaire).
BUILDERS	27	page -> fabrique (eleves, caisse, classes, notes, planning, personnel, parametres, comptes).

Classe `MainWindow(QMainWindow)` (ligne 39) :

Methode	Ligne	Role
<code>__init__(user)</code>	40	Stocke l'utilisateur, cree <code>PageContext</code> , applique <code>main.ui</code> , remplit les labels, cree le label de statut API, cable la navigation, construit les pages, navigue vers <code>dashboard</code> .
<code>_refresh_api_status()</code>	106	Interroge <code>api.api_disponible()</code> et met a jour le label de la barre d'etat.
<code>logout()</code>	123	Confirme puis ferme la fenetre.
<code>_wire_nav()</code>	128	Affiche les boutons autorises, masque les autres.
<code>_build_pages()</code>	138	Construit les pages autorisees dans des <code>QScrollArea</code> ; toute erreur affiche un libelle rouge.
<code>navigate(page_name)</code>	173	Bascule la page, appelle <code>refresh()</code> , coche le bouton.

9.3 `views/login_view.py` (125 lignes)

Element	Ligne	Role
<code>PRIMARY / PRIMARY_DARK</code>	11-12	Couleurs du bouton de connexion (<code>#047857</code> / <code>#065f46</code>).
<code>_Gradient(QFrame)</code>	15	Arriere-plan en degrade peint via <code>QPainter.paintEvent</code> .
<code>LoginDialog(QDialog)</code>	25	Dialogue de connexion 700x540.
<code>_build()</code>	34	Construit l'interface en code : carte blanche arrondie, champs nom/mot de passe, bouton degrade, rappel des comptes de test.
<code>_do_login()</code>	112	Appelle <code>auth.login</code> , affiche l'erreur si besoin, sinon stocke <code>self.user</code> et <code>accept()</code> .

9.4 `views/pages.py` (1455 lignes) — les pages metier

C'est le cœur de l'interface. Chaque fabrique suit le meme contrat : `fabrique(page, ctx)` remplit le widget `page` , connecte les signaux et expose `page.refresh` .

Contexte commun

Element	Ligne	Role
<code>PageContext</code>	22	Wrapper : <code>user</code> , <code>role</code> , <code>authorizer</code> , <code>navigate</code> , <code>can_edit(page)</code> .

Utilitaires

Fonction	Ligne	Role
<code>_btn(text, callback, style)</code>	33	Bouton compact (hauteur max 28 px, curseur main).
<code>_simple_btn_style(...)</code>	42	Style de bouton discret (fond clair, bordure).
<code>_money_edit(...)</code>	46	Champ montant en FCFA.
<code>_today_fr()</code>	55	Date du jour en francais.
<code>_fill_combos(combo, items)</code>	62	Remplit une combo.

Dashboards

Fabrique	Ligne	Role
<code>dashboard_admin</code>	69	KPIs des comptes (actifs, inactifs), dernieres connexions, actions rapides (nouveau compte, reinitialiser un mot de passe).
<code>dashboard_directeur</code>	99	Masse salariale, enseignants (API), taux de scolarite (API), graphiques (tresorerie 6 mois, effectifs par classe, personnel), validations (classes sans titulaire -> double-clic), actions (paie, rapport RH, auditer la caisse, recrutement).
<code>dashboard_gestionnaire</code>	204	KPIs (eleves, inscriptions du jour, encaissements du jour, dossiers incomplets), activite recente, dossiers incomplets, graphique des statuts, actions rapides (inscrire, recette, depense, certificat).
<code>_directeur_charts()</code>	175	Donnees des graphiques du directeur (6 mois, 6 premieres classes).
<code>_replace_layout(layout, widget)</code>	195	Remplace le contenu d'un layout par un graphique.

Les KPIs des dashboards utilisent l'API **quand elle est disponible** (ex. `total_eleves`, `total_montant_paiement`, `total_enseignant`), sinon les valeurs de la base locale.

Page Eleves

Element	Ligne	Role
<code>eleves(page, ctx)</code>	269	Tableau des eleves. Par default, une classe est preselectionnee (la premiere) ; l'option « Toutes les classes » reste disponible. Filtres classe/statut/recherche, boutons Modifier/ Supprimer par ligne, double-clic pour modifier, export CSV, 4 KPIs recalculés sur la classe choisie.
<code>open_inscription_dialog(parent, ctx, eleve=None)</code>	348	Dialogue d'inscription (1000x780) : aperçu du matricule, reinscription par recherche de matricule, classe (creation a la volée), dossier complet (naissance, parents, adresse, documents), montant verse + mode de

Element	Ligne	Role
		reglement (encaisse automatiquement, reçu imprimable).
<code>_parse_money(text)</code>	531	Parse un montant saisi (espaces, virgules, « FCFA »).

Page Classes

Element	Ligne	Role
<code>classes(page, ctx)</code>	538	Tableau des classes (nom, effectif, capacite, titulaire, salle) ; alerte rouge quand la classe est pleine ; recherche ; 4 KPIs.
<code>open_classe_dialog(parent, ctx, classe=None, on_created=None)</code>	606	Creer/modifier une classe (nom, niveau, capacite, salle, titulaire).

Page Notes

Element	Ligne	Role
<code>notes(page, ctx)</code>	654	Saisie par classe x matiere x periode : tableau editable, moyenne $(d1+d2+2 \times comp)/4$ et appreciation calculees en direct, bornage 0-20, ajout de matiere, generation des bulletins. Les boutons de saisie sont masques si le role ne peut pas saisir.
<code>_appreciation(moyenne)</code>	800	Mention locale (identique a <code>reports._appreciation</code>).

Page Planning

Element	Ligne	Role
<code>PlanningCellDialog</code>	813	Dialogue pour affecter matiere + salle a une case.
<code>planning(page, ctx)</code>	845	Grille jours (6 colonnes) x creneaux (8 lignes). Mode edition : double-clic pour affecter, « Enregistrer le Planning » reecrit tout. Bouton d'impression -> HTML.

Page Caisse

Element	Ligne	Role
<code>caisse(page, ctx)</code>	959	Tableau des transactions filtre par type, recherche et dates (mois courant par default). Colonnes Entree/Sortie separees, total entrees, total sorties, solde. Export CSV. Boutons Recette/Depense masques si le role ne saisit pas.
<code>open_transaction_dialog(parent, ctx, type_trans)</code>	1037	Saisie motif, beneficiaire, montant, categorie, mode de reglement.

Page Comptes (admin)

Element	Ligne	Role
<code>comptes(page, ctx)</code>	1083	

Element	Ligne	Role
		Tableau des comptes (nom, email, role, actif), filtres, actions par ligne (Activer/ Desactiver, Mdp, Supprimer), 4 KPIs.
<code>open_compte_dialog(parent, ctx, compte=None)</code>	1148	Creer/modifier un compte ; mot de passe temporaire genere et affiche.
<code>open_change_password_dialog(parent, user)</code>	1194	Changer son propre mot de passe.
<code>open_reset_password_dialog(parent, ctx)</code>	1223	Reinitialiser le mot de passe d'un compte non-admin.

Page Personnel

Element	Ligne	Role
<code>personnel(page, ctx)</code>	1249	Tableau du personnel construit en code (nom, fonction, telephone, email, salaire), recherche, « + Nouvel Employe ».
<code>open_personnel_dialog(parent, ctx, employe=None)</code>	1320	Creer/modifier un employe (fonction, salaire, statut Contrat/CDI/Vacataire/Stage).

Certificat

Element	Ligne	Role
<code>open_certificat_dialog(parent)</code>	1366	Dialogue filtre par classe : choisir la classe puis l'eleve, et genere le certificat de scolarite.

Parametres

Element	Ligne	Role
<code>parametres(page, ctx)</code>	1403	Signataire (nom/titre), ville, pays + images (bandeau haut/bas, signature) copiees dans <code>DOCS_DIR</code> . Bouton supprimer la configuration.

9.5 `views/widgets.py` (176 lignes) — graphiques et formats

Element	Ligne	Role
<code>CHART_COLORS</code>	8	Palette de couleurs des graphiques.
<code>_color(i)</code>	13	Couleur par index (cycle sur la palette).
<code>fmt_money(montant)</code>	17	1 234 567 FCFA (separateur d'espace).
<code>fmt_money_short(montant)</code>	24	Sans le suffixe « FCFA ».
<code>_BaseChart(QWidget)</code>	28	Base commune des graphiques (marge, titre, anti-aliasing).
<code>SimpleBarChart</code>	70	Histogramme (valeurs au-dessus des barres, echelle Y). <code>set_data(labels, values)</code> .
<code>SimplePieChart</code>	117	Camembert (pourcentages, legende). <code>set_data(labels, values)</code> .

10. Fichiers .ui

Les interfaces sont definies dans `views/ui_files/` et chargees par `apply_ui`. Les fichiers reellement utilises :

Fichier	Charge par	Role
<code>main.ui</code>	<code>main_view.py:83</code>	Fenetre principale (sidebar + stackedWidget + labels + bouton deconnexion).
<code>dashboards/dashboard_admin.ui</code>	<code>pages.py:70</code>	Accueil admin.
<code>dashboards/dashboard_directeur.ui</code>	<code>pages.py:100</code>	Accueil directeur.
<code>dashboards/dashboard_gestionnaire.ui</code>	<code>pages.py:205</code>	Accueil gestionnaire.
<code>eleves/eleves.ui</code>	<code>pages.py:270</code>	Liste des eleves.
<code>eleves/inscription.ui</code>	<code>pages.py:352</code>	Dossier d'inscription.
<code>classes/classes.ui</code>	<code>pages.py:539</code>	Liste des classes.
<code>classes/classe_dialog.ui</code>	<code>pages.py:610</code>	Dialogue classe.
<code>notes/notes.ui</code>	<code>pages.py:655</code>	Saisie des notes.
<code>planning/planning.ui</code>	<code>pages.py:846</code>	Emploi du temps.
<code>caisse/caisse.ui</code>	<code>pages.py:960</code>	Caisse.
<code>comptes/comptes.ui</code>	<code>pages.py:1084</code>	Liste des comptes.
<code>comptes/compte_dialog.ui</code>	<code>pages.py:1152</code>	Dialogue compte.
<code>parametres/parametres.ui</code>	<code>pages.py:1404</code>	Parametres des documents.

11. Base de donnees SQLite : schema complet

Fichier : `data/ecole.db`. Cree par `SCHEMA` (`db.py:7`).

Tables

utilisateurs — comptes de connexion

Colonne	Type	Contrainte
<code>id</code>	INTEGER	PK AUTOINCREMENT
<code>nom_complet</code>	TEXT	NOT NULL
<code>username</code>	TEXT	NOT NULL UNIQUE
<code>email</code>	TEXT	—
<code>telephone</code>	TEXT	—
<code>password</code>	TEXT	NOT NULL (hash sha256)
<code>role</code>	TEXT	NOT NULL
<code>actif</code>	INTEGER	NOT NULL DEFAULT 1
<code>created_at</code>	TEXT	DEFAULT datetime('now','localtime')

Colonne	Type	Contrainte
last_login	TEXT	—

classes : id, nom (NOT NULL), niveau, capacite (INTEGER DEFAULT 50), salle, titulaire

matieres : id, nom (TEXT NOT NULL UNIQUE), coefficient (REAL DEFAULT 1)

elevés : id, matricule (UNIQUE), nom (NOT NULL), prenom (NOT NULL), sexe, date_naissance, lieu_naissance, classe_id (FK -> classes), ecole_provenance, pere_nom, pere_tel, mere_nom, mere_tel, tuteur_nom, tuteur_tel, adresse, check_acte / check_photos / check_bulletin (INTEGER DEFAULT 0), statut (TEXT DEFAULT 'Inscrit'), date_inscription (TEXT DEFAULT date('now','localtime'))

notes : id, eleve_id (FK), matiere_id (FK), periode (TEXT), devoir1 / devoir2 / composition (REAL), **UNIQUE(eleve_id, matiere_id, periode)**

transactions : id, date (DEFAULT aujourd'hui), reference (NOT NULL), beneficiaire, motif, categorie, montant (REAL NOT NULL), type (CHECK IN ('entree','sortie')), mode_reglement

planning : id, classe_id (FK), jour (TEXT), creneau (TEXT), matiere, salle, **UNIQUE(classe_id, jour, creneau)**

connexions : id, utilisateur_id (FK), date_connexion (DEFAULT maintenant) -> journal d'audit des connexions.

personnel : id, nom_complet (NOT NULL), fonction, telephone, email, salaire (REAL DEFAULT 0), statut (DEFAULT 'Contrat')

parametres : cle (TEXT PK), valeur (TEXT) -> paires cle/valeur (signataire_nom, signataire_titre, ville, pays, frais_scolarite, bandeau_haut, bandeau_bas, signature).

Comptes par défaut (creees au premier lancement)

Nom	Identifiant	Mot de passe	Role
Administrateur Systeme	admin	admin123	admin
Directeur de l'Ecole	directeur	directeur123	directeur

Nom	Identifiant	Mot de passe	Role
Gestionnaire Scolaire	gestionnaire	gestionnaire123	gestionnaire

A changer des la mise en production réelle.

12. Roles et permissions

Action	admin	directeur	gestionnaire
Voir le dashboard	OUI (comptes)	OUI	OUI
Gerer les comptes	OUI	NON	NON
Gerer les eleves	NON	consulter	saisir
Gerer les classes	NON	consulter	saisir
Saisir les notes	NON	NON	OUI
Gerer le planning	NON	consulter	saisir
Caisse (recettes/depenses)	NON	NON	OUI
Personnel / RH	NON	OUI (paie, rapports)	NON
Parametres des documents	NON	OUI	NON

- Navigation : `RoleAuthorizer.NAV` (auth.py:48) — les boutons de la sidebar sont masques selon le role (main_view.py:128).
- Saisie : `RoleAuthorizer.can_edit` (auth.py:64) — les boutons d'ajout et de modification sont masques quand le role ne peut pas saisir.

13. Construction des executables

Executable Linux

```
python3 -m venv venv
venv/bin/pip install -r requirements.txt
venv/bin/pyinstaller --noconfirm --clean --onefile --windowed \
    --name GestionScolaire \
    --add-data "views/ui_files:views/ui_files" \
    main.py
```

Sortie : `dist/GestionScolaire` (one-file, sans console). Les `.ui` sont embarques via `--add-data`. Hidden imports : `PyQt5.uic`, `PyQt5.uic.plugins`, `PyQt5.QtSql`.

Executable Windows (via GitHub Actions)

Le workflow `.github/workflows/build_windows.yml` construit l'exé Windows sur chaque `push` vers la branche `exe` :

1. `checkout` du code ;
2. `setup-python 3.11` ;
3. installation de `requirements.txt` + `pyinstaller` ;
4. `pyinstaller --noconfirm --clean build_win.spec` (one-file, sans console) ;
5. dépose de `dist/GestionScolaire.exe` comme **artefact** de la run.

L'artefact est téléchargeable depuis la page Actions du dépôt GitHub (Actions -> Build Windows Executable -> Artifact).

Spec Windows (`build_win.spec`)

`build_win.spec` construit un **one-file** nommé `GestionScolaire.exe`, sans console, en embarquant `views/ui_files` et le manifeste DPI.

Manifeste DPI Windows (`win_dpi_manifest.xml`)

Le manifeste déclare `<dpiAware>true</dpiAware>`. Il est **embarqué dans l'exé** par `build_win.spec` (`EXE(manifest=...)`). Combine au `_setup_high_dpi` de `main.py`, il rend le texte net sur les écrans Windows 100/125/150/200 %.

Le fichier n'est pas signé : Windows SmartScreen peut afficher un avertissement au premier lancement (« Informations supplémentaires -> Exécuter quand même »).

Vérification après build

- Lancer l'exécutible : il doit rester vivant et créer `data/ecole.db`.
- Windows : tester avec `QT_SCALE_FACTOR=1.25` pour simuler un écran 125 %.

14. Guide d'utilisation par role

Connexion

Identifiants de test : admin/admin123 , directeur/directeur123 , gestionnaire/gestionnaire123 (rappelles sur l'ecran de connexion).

Gestionnaire (saisie quotidienne)

1. **Inscription** : dashboard -> « Inscrire » -> remplir le dossier (nom, prenom obligatoires, classe obligatoire) -> montant verse eventuel -> reçu imprimable. Reinscription : saisir le matricule -> « Rechercher ».
2. **Eleves** : la liste s'ouvre sur la **premiere classe** ; changer la classe avec la liste deroulante ou choisir « Toutes les classes ». Les chiffres (KPIs) se mettent a jour sur la classe choisie.
3. **Classes** : creer/modifier (nom, niveau, capacite, salle, titulaire).
4. **Notes** : choisir classe, matiere, periode -> « Charger » -> saisir -> « Enregistrer les notes ». Moyenne et appreciation calculees en direct.
5. **Planning** : choisir la classe -> « Modifier » -> double-cliquer sur une case pour affecter matiere/salle -> « Enregistrer le Planning ».
6. **Caisse** : « + Recette » / « + Depense ». Filtres par type, texte et dates. Export CSV.
7. **Certificat** : dashboard -> « Certificat » -> choisir la **classe** puis l'**eleve** -> « Generer ».

Directeur (supervision)

- Dashboard : masse salariale, taux de scolarite, graphiques, validations (classes sans titulaire -> double-clic pour affecter), actions « Paie », « Rapport RH », « Auditer la caisse ».
- Consulte eleves/classes/notes/planning/caisse/personnel/parametres **sans pouvoir saisir** (les boutons d'edition sont masques).
- Parametres : renseigne le signataire (nom/titre), la ville, le pays et les images d'en-tete/signature des documents.

Admin (comptes uniquement)

- Dashboard : comptes actifs/inactifs, dernieres connexions.

- Page Comptes : creer un compte (mot de passe temporaire affiche), activer/desactiver, reinitialiser le mot de passe, supprimer.
- Ne voit ni les eleves, ni la caisse, ni les notes (sidebar reduite).

Documents generes (HTML ouverts dans le navigateur)

Bulletins (page Notes), reçu de paiement (inscription), certificat de scolarite, paie + rapport RH (dashboard directeur), emploi du temps (planning). Les fichiers sont stockes dans `data/documents/`.

15. Depannage et questions frequentes

Probleme	Cause / solution
L'executable ne s'ouvre pas / crashe silencieusement	Lancer depuis un terminal pour voir l'erreur, ou lire la boite « Erreur inattendue » du <code>_excepthook</code> .
L'API FastAPI n'est pas lancee	L'app affiche « API : hors ligne (mode local) » dans la barre d'etat et fonctionne quand meme sur la base locale.
Les chiffres du dashboard semblent differents	Sans API, les indicateurs viennent de la base locale ; avec API, ils viennent du serveur. C'est le comportement attendu.
« Module not found » en build	hiddenimports necessaires : <code>PyQt5.uic</code> , <code>PyQt5.uic.plugins</code> , <code>PyQt5.QtSql</code> .
Texte flou sur Windows	Verifier que le manifeste est embarque et que <code>_setup_high_dpi</code> est execute avant <code>QApplication</code> .
Boutons / champs geants	Ne jamais forcer <code>QSizePolicy.Expanding</code> sur les enfants d'une page (voir <code>_make_responsive</code>).
Base ecrasee / perdue	<code>data/ecole.db</code> vit a cote de l'exe. Sauvegarder ce fichier.
Fichier CSV illisible dans Excel	Le CSV utilise ; et un BOM (<code>utf-8-sig</code>) — compatible Excel francophone.
SmartScreen bloque l'exe Windows	L'exe n'est pas signe : « Informations supplementaires -> Executer quand meme ».
Documents HTML generes	Ils s'ouvrent dans le navigateur par default ; ranges dans <code>data/documents/</code> .

16. Historique des versions

Version	Contenu
1.0.x	Prototype : API FastAPI + prototype web.
1.1.x	Bascule desktop PyQt5 + SQLite, architecture a 3 couches.
1.2.x	Version actuelle : pages metier completes, dashboards par role, client API FastAPI avec repli local, filtre des eleves par classe par default, certificat filtrable, exe

Version	Contenu
	Windows construit via GitHub Actions, exe Linux one-file. Code nettoye : tous les docstrings supprimes, commentaires courts en francais.

Fin de la documentation. Les references de lignes correspondent au code source de la version 1.2.0 (branche `exe`).